

A New Perspective for Shuttlecock Hitting Event Detection

Yu-Hsi Chen

Institute of Communications Engineering, National Tsing Hua University, Taiwan
yuhsi44165@gapp.nthu.edu.tw

Abstract

This article introduces a novel approach to shuttlecock hitting event detection. Instead of depending on generic methods, we capture the hitting action of players by reasoning over a sequence of images. To learn the features of hitting events in a video clip, we specifically utilize a deep learning model known as SwingNet. This model is designed to capture the relevant characteristics and patterns associated with the act of hitting in badminton. By training SwingNet on the provided video clips, we aim to enable the model to accurately recognize and identify the instances of hitting events based on their distinctive features. Furthermore, we apply the specific video processing technique to extract the prior features from the video, which significantly reduces the learning difficulty for the model. The proposed method not only provides an intuitive and user-friendly approach but also presents a fresh perspective on the task of detecting badminton hitting events. The source code will be available at <https://github.com/TW-yuhsi/A-New-Perspective-for-Shuttlecock-Hitting-Event-Detection>.

1 Introduction

In this task, we need to develop computer vision technology capable of automatically extracting comprehensive technical data from the broadcast video of badminton matches. This data includes crucial information about each shot, such as the precise timing, spatial positioning, player postures, and skill levels exhibited as the shuttlecock is struck during the match. It encompasses various elements, such as the shot's timing, ball's location, player responsible for the hit, swing posture, standing positions of both players, type of ball used, and the eventual winner. The final rankings are determined by the total scores, with the highest score securing the top position and the lowest score occupying the last spot.

2 Methodology

Various methods have been utilized in the task, including video processing techniques, SwingNet, ViT, YOLOv5, YOLOv8, and TrackNetV2 deep learning models.



Figure 1: Comparison between the original and optical frames.

2.1 Video Processing

Video processing is a critical step to improve the score in the entire task. In this process, we first calculate the optical flow images using the method of Optical Flow Calculation embedded in Reynolds Transport Theorem¹ [Bruhn *et al.*, 2005; Harouna and Mémin, 2017] to capture the prior features of the video, then, we remove the background information to achieve a mechanism similar to attention as shown in Figure 1. And we will use the term "optical flow video without background" to specifically describe this type of video. Subsequently, we input these processed video into SwingNet for shuttlecock hitting event detection.

2.2 SwingNet

SwingNet [McNally *et al.*, 2019] plays a crucial role in the field of computer vision and sports analytics, specifically in the domain of golf. Its application enables more efficient and detailed analysis of golf swings, contributing to the overall development and understanding of the sport. In here, we utilize the SwingNet, a deep learning model combining with the MobileNetV2 and bidirectional LSTM structure, for shuttlecock hitting event detection. That is, utilizing SwingNet enables us to extract the ShotSeq and HitFrame features for the desired csv file. Figure 2 demonstrates the inferred event probabilities for a video clip.

¹A Prior Feature Enhanced Layer for Small Objects Detection and Tracking

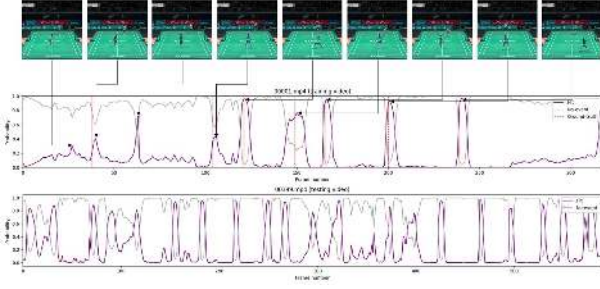


Figure 2: SwingNet performs event probability inference.

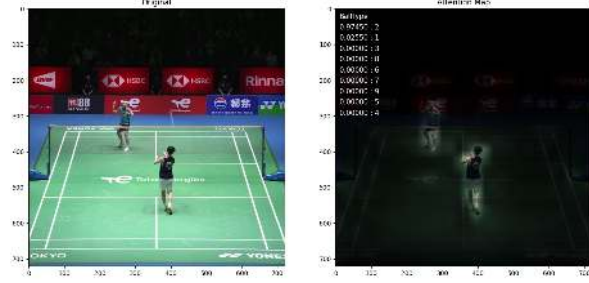


Figure 3: Attention map for BallType classification task.

2.3 Vision Transformer

ViT, short for Vision Transformer [Dosovitskiy *et al.*, 2020], is a deep learning model that applies the transformer architecture to computer vision tasks. ViT adopts the strategy by leveraging the transformer architecture, initially designed for tasks in natural language processing. Moreover, the strength of ViT lies in its attention mechanism, as depicted in Figure 3. Here, we utilize ViT-B/16 to extract the information including Hitter, RoundHead, Backhand, BallHeight, BallType, and Winner from the videos.

2.4 YOLOv5

The YOLO-series detector is a renowned family of object detection models. Specifically, YOLOv5 [Zhu *et al.*, 2021] is a popular and highly efficient object detection model, which builds upon the success of its predecessors, YOLOv1, YOLOv2, and YOLOv3, and introduces several improvements to achieve better performance in terms of accuracy and speed. We utilize YOLOv5m to extract the information for LandingX and LandingY. Once we have obtained the detection results for the ball, we integrate them with the Hitter prediction generated by ViT. We designate the output generated by ViT as the hitter, while the remaining player is assigned the role of the defender. As the landing result of the ball is the desired information, we utilize the y-value of the bottom-right corner of hitter’s bounding box to represent the ball’s landing y-value. The landing coordinate of the ball is indicated by the black cross symbol depicted in Figure 4.

2.5 TrackNetV2

The intention behind incorporating TrackNetV2 in this context is to compensate for the shortcomings of YOLO in de-



Figure 4: Detection results from YOLOv5m and YOLOv8-pose.

tecting badminton. In addition, I will assign the coordinates of LandingX and LandingY as (0, 0) if both YOLOv5m and TrackNetV2 fail to detect the shuttlecock simultaneously.

2.6 YOLOv8-pose

YOLOv8 [Jocher *et al.*, 2023] is the latest version of YOLO by Ultralytics. Being at the forefront of advanced technology, YOLOv8 represents a SOTA model that builds upon the achievements of its predecessors. It incorporates innovative features and enhancements to deliver superior performance, versatility, and efficiency. Furthermore, it provides comprehensive support for a wide array of vision AI tasks, encompassing detection, segmentation, pose estimation, tracking, and classification. Here, we utilize the pose estimated by YOLOv8x-pose-p6 model to acquire the players’ foot positions with enhanced accuracy as depicted in Figure 4.

3 Results and Discussion

This section encompasses the Evaluation Criteria, Experimental Results, and Discussion.

3.1 Evaluation Criteria

The scoring system is based on each individual video clip, with a maximum score of 1 point per clip. The first step is to validate the number of shots within the rally video, with each row in the corresponding csv file representing one shot. If the predicted number of shots differs from the ground truth, the question will receive a score of 0 points. However, if the number of shots is accurately predicted, a score of 0.1 points will be awarded, and the evaluation will proceed to the next stage, which involves comparing the contents of the columns. In this stage, the maximum score for correctly matching column content is 0.9 points.

When the number of shots is accurate for the test video, each shot is evaluated individually, and its score is determined based on the cumulative score of its columns. The average score of all the columns is then used as the shot’s content score. The evaluation process begins by validating the Hit-Frame column. If the prediction error exceeds 2 frames compared to the true value, the shot is awarded 0 points in terms of content and the scoring proceeds to the next shot. However, if the prediction error is within or equal to 2 frames, a score of 0.1 points is given, and the remaining columns within

the same shot are compared. Assuming that there are R rally videos in the data set, and the i -th video has S_i shots, the scoring formula is given below:

$$\frac{1}{R} \sum_{i=1}^R 1_{S_i=S_i^{pred}} (0.1 + ASS_i) \quad (1)$$

in which ASS_i (the Average Shot Score of content of the i -th rally video) is given by:

$$ASS_i = \frac{1}{S_i} \sum_{j=1}^{S_i} 1_{|HitFrame_j - HitFrame_j^{pred} \leq 2} SS_j \quad (2)$$

with SS_j (the j -th Shot Score) given by:

$$\begin{aligned} SS_j = & 0.1 + 0.1 \times 1_{Hitter_j = Hitter_j^{pred}} \\ & + 0.1 \times 1_{BallHeight_j = BallHeight_j^{pred}} \\ & + 0.1 \times 1_{\|Landing_j - Landing_j^{pred}\| < 6} \\ & + 0.05 \times 1_{\|HitterLocation_j - HitterLocation_j^{pred}\| < 10} \\ & + 0.05 \times 1_{\|DefenderLocation_j - DefenderLocation_j^{pred}\| < 10} \\ & + 0.05 \times 1_{Backhand_j = Backhand_j^{pred}} \\ & + 0.05 \times 1_{RoundHead_j = RoundHead_j^{pred}} \\ & + 0.2 \times 1_{BallType_j = BallType_j^{pred}} \\ & + 0.1 \times 1_{Winner_j = Winner_j^{pred}} \end{aligned} \quad (3)$$

3.2 Experimental Results

To begin with, we will discuss the hyperparameters and performance of SwingNet, as this aspect determines the upper limit of the overall score. This is due to the fact that any errors in the detection of ShotSeq and HitFrame would render subsequent feature accuracy irrelevant, as they would not contribute to the score. Subsequently, we will delve into the hyperparameters and performance analysis of the ViT-B/16 and YOLOv5m models. Consequently, the contribution of each model to the score will be illustrated at the conclusion of this section.

Hyperparameters and Performance of SwingNet

To enhance the performance of SwingNet in capturing key frames more effectively, we conducted numerous experiments focused on optimizing the hyperparameters. After lots of trials, we have determined the hyperparameters that lead to improved predictions for the model as shown in Table 1.

Once the hyperparameters of SwingNet have been determined, it becomes essential to examine how various video representations influence the detection results. We utilized three distinct types of input videos, including original video, optical flow video (Opt video), and optical flow video without background (Opt video w/o BG). Based on the information presented in Table 2, it can be observed that utilizing the optical flow video without background yields the highest score compared to other types of videos. Therefore, we used optical flow without background as the main type of video for

Hyperparameters	Train	Inference
image size	180×180	180×180
sequence length	64	64
number of frozen layers	10	10
batch size	8	8
event ratio	0.0305	—
iteration number	10000	—
probabilities for the quantiles	—	0.8
filter threshold	—	3

Table 1: Optimized hyperparameters for SwingNet.

Video type	Iter. \ Loss	Score
Original video	2000 \ 0.5821	0.0234
	4000 \ 0.5074	0.0275
	6000 \ 0.4622	0.0184
	8000 \ 0.4269	0.0276
	10000 \ 0.3979	0.0239
Opt video	2000 \ 0.3799	0.0229
	4000 \ 0.3325	0.0325
	6000 \ 0.3037	0.0333
	8000 \ 0.2839	0.0249
	10000 \ 0.2664	0.0334
Opt video w/o BG	2000 \ 0.3808	0.0274
	4000 \ 0.3294	0.0331
	6000 \ 0.2982	0.0349
	8000 \ 0.2760	0.0225
	10000 \ 0.2582	0.0263

Table 2: Performance of SwingNet on different types of videos.

detection, and performed 5-fold inference. All the scores for the 5-fold inference are presented in Table 3.

After multiple trials, the highest score obtained in the first two columns, as shown in Table 3, is 0.0426. In addition, we attempted to ensemble the detection results of SwingNet, but it did not effectively improve the score. Furthermore, it is notable that the maximum score for the first two columns can reach 0.2 according to the evaluation criteria. This observation further emphasizes that SwingNet’s predictions for the first two columns still have great potential for improvement.

Upon completing the first two columns, we will exhibit the hyperparameters employed for classification and detection tasks using ViT-B/16 and YOLOv5m, respectively.

Hyperparameters of ViT-B/16 and YOLOv5m

Based on numerous experiments conducted, we have determined the optimal hyperparameters that improve the performance of ViT-B/16 and YOLOv5m, as presented in Table 4. To enhance the inference performance of the ViT-B/16 and YOLOv5m models, the training strategy involves maximizing the image size. The main reason behind this setting is that larger image sizes have a tendency to uncover finer details, thereby significantly enhancing the value of both training and inference processes. In addition, I divided the training data into 5-fold for the classification tasks. Also, it can be

Iter.\fold	1	2	3	4	5
1000	0.0259	0.0262	0.0168	0.027	0.0263
2000	0.0241	0.0261	0.0354	0.034	0.0339
3000	0.0212	0.0352	0.037	0.0318	0.0426
4000	0.0275	0.0266	0.0389	0.0372	0.0399
5000	0.0288	0.027	0.0289	0.0366	0.0351
6000	0.0312	0.0167	0.0362	0.0269	0.0283
7000	0.0233	0.0372	0.0144	0.0326	0.0319
8000	0.0285	0.0347	0.0375	0.0396	0.0213
9000	0.0236	0.0237	0.0387	0.0279	0.0226
10000	0.0324	0.0182	0.0348	0.0262	0.0251

Table 3: Scores obtained from 5-fold inference.

Hyperparameters	ViT-B/16	YOLOv5m
image size	480	2880
optimizer	SGD	SGD
learning rate	3E-2	1E-2
loss function	CE	BCE
batch size	4	1
iteration number	10000	–
epochs	–	100

Table 4: Hyperparameters of ViT-B/16 and YOLOv5m.

observed that there is only a slight difference between with and without using the ensemble techniques. Furthermore, the variations among different ensemble techniques are also small as shown in Table 5.

Now, let’s analyze the individual contributions of different models to their respective features. Firstly, we will use SwingNet to obtain the predicted answers for ShotSeq and HitFrame. Subsequently, we utilize ViT-B/16 for predicting the features related to the classification task. Furthermore, we employ the YOLOv5m model with TrackNetV2 to detect the position of the players and the trajectory of the badminton. Moreover, we utilized the YOLOv8x-pose-p6 model to estimate the poses of players, resulting in a more accurate detection of their positions as shown in Figure 4. These steps allow us to generate the complete submission file. The corresponding contribution table can be found in Table 6. Lastly, the private score in final phase this task is 0.3483.

Features\Ensemble	fold1	Vote	Mean
Hitter	+0.0068	+0.0068	+0.0068
RoundHead	+0.0033	+0.0033	+0.0033
Backhand	+0.0031	+0.0031	+0.0031
BallHeight	+0.0066	+0.0066	+0.0067
BallType	+0.0089	+0.0091	+0.0089
Winner	+0.0005	+0.0005	+0.0006

Table 5: Ablation study of ensemble techniques.

Features	Models	Score
ShotSeq	SwingNet	0.0426
&HitFrame		
Hitter	ViT	0.0494 (+0.0068)
RoundHead	ViT	0.0527 (+0.0033)
Backhand	ViT	0.0558 (+0.0031)
BallHeight	ViT	0.0625 (+0.0067)
LandingX	YOLOv5m	0.0625 (+0.0000)
&LandingY	+TrackNetV2	
HitterLocationX	YOLOv5m	0.0630 (+0.0005)
&HitterLocationY		
&DefenderLocationY		
&DefenderLocationY	+YOLOv8x	-pose-p6
BallType	ViT	
Winner	ViT	0.0721 (+0.0091)
		0.0727 (+0.0006)

Table 6: The distinct contributions of various models to the features.

3.3 Discussion

In this section, we will primarily address four key points of discussion. The first one is the extensive GPU memory resources required when using the method proposed in this article for shuttlecock hitting event detection with SwingNet. The main reason is that the model reads in 64 frames of video at once for evaluation. The second point is that, according to the rules of badminton, we can infer that the information in the Hitter column will alternate between A and B ($A \rightarrow B \rightarrow A \rightarrow B \dots$). This implies that there will never be more than two consecutive instances of either A or B .

The third point to consider is that by exclusively focusing on the hitter during the classification process, it is possible to enhance the performance of the classification problem by classify the local area instead of utilizing the entire image for classification as depicted in Figure 3. Implementing this method has the potential to decrease the defender’s propensity to confuse hitter’s information. Lastly, it is worth noting that in this competition, several features were not utilized while training, such as LandingX, LandingY, HitterLocationX, HitterLocationY, DefenderLocationX, DefenderLocationY. Without fully utilizing the available information, I believe there will be a certain degree of misalignment.

4 Summary

This article proposes a new perspective for the shuttlecock hitting event detection task, which involves using a deep learning model to learn the meaning conveyed by the image sequence, and thus capturing the desired information from the video. Furthermore, the video processing method presented in this article can effectively reduce the difficulty of model learning, resulting in better performance in keyframe detection task. In the future, we will continue to explore adjustments to the model architecture in order to construct a more precise method for obtaining keyframes. This will be combined with suitable image processing techniques to further enhance the overall framework.

References

- [Bruhn *et al.*, 2005] Andrés Bruhn, Joachim Weickert, and Christoph Schnörr. Lucas/kanade meets horn/schunck: Combining local and global optic flow methods. *International journal of computer vision*, 61:211–231, 2005.
- [Dosovitskiy *et al.*, 2020] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [Harouna and Mémin, 2017] S Kadri Harouna and Etienne Mémin. Stochastic representation of the reynolds transport theorem: revisiting large-scale modeling. *Computers & Fluids*, 156:456–469, 2017.
- [Jocher *et al.*, 2023] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. YOLO by Ultralytics, January 2023.
- [McNally *et al.*, 2019] William McNally, Kanav Vats, Tyler Pinto, Chris Dulhanty, John McPhee, and Alexander Wong. GolfdB: A video database for golf swing sequencing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 0–0, 2019.
- [Zhu *et al.*, 2021] Xingkui Zhu, Shuchang Lyu, Xu Wang, and Qi Zhao. Tph-yolov5: Improved yolov5 based on transformer prediction head for object detection on drone-captured scenarios. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2778–2788, 2021.